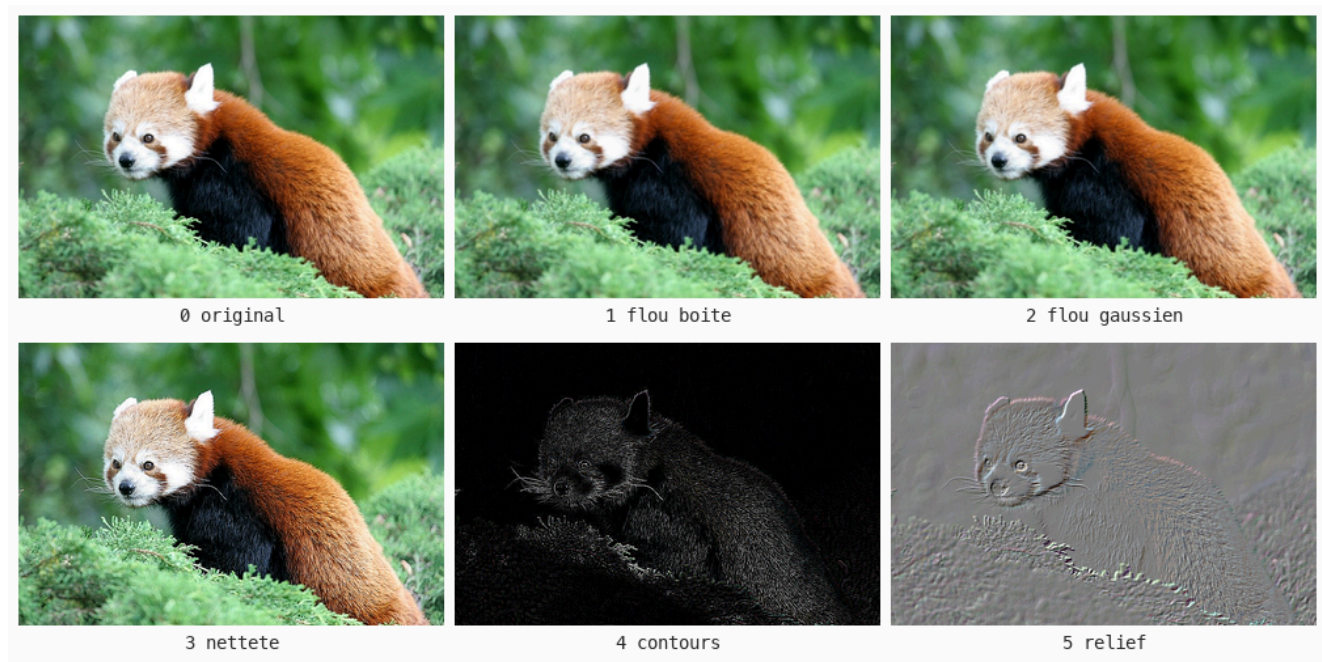


Cours 17 — La convolution : un pixel et ses voisins

Avant : cours 11 (double boucle source / résultat), cours 14. **Comment travailler** : toujours dans ofApp.h et ofApp.cpp , par versions successives. ofApp17.h / ofApp17.cpp : la référence téléchargeable de l'état final. pandaroux.jpg va dans bin/data .

Au cours 11, un pixel ne dépendait que de lui-même ; au 14, d'un autre pixel. Ici, il dépend de **neuf** : lui et ses huit voisins. Flou, netteté, contours, relief — un seul code, seule une grille de neuf nombres change. C'est la **convolution**, l'outil central du traitement d'image.



1. Le noyau

pixel et ses 8 voisins noyau (poids) nouveau pixel

120	130	125
118	140	128
122	126	124

×

0	-1	0
-1	5	-1
0	-1	0

=

198

somme = $140 \times 5 - 130 - 118 - 128 - 126 = 198$ (le centre ressort : plus net)
chaque voisin est multiplié par son poids, on additionne tout

Un **noyau** est une grille 3×3 de poids. Pour calculer un pixel du résultat, on prend les 3×3 pixels de la source centrés au même endroit, on multiplie chaque pixel par le poids qui lui fait face, et on additionne

tout. C'est une **moyenne pondérée** des voisins.

Noyau	Somme des poids	Effet
tous à 1/9	1	flou boîte : la moyenne des neuf
1 2 1 / 2 4 2 / 1 2 1, divisé par 16	1	flou gaussien : le centre pèse plus, flou plus doux
0 -1 0 / -1 5 -1 / 0 -1 0	1	netteté : le centre renforcé, les voisins retirés
-1 partout, 8 au centre	0	contours : une zone uniforme donne 0, seules les différences restent
-1 en haut à gauche, 1 en bas à droite	0	relief : différence entre deux coins opposés

Quand la somme vaut 1, la luminosité globale est conservée. Quand elle vaut 0, une zone plate devient noire : c'est ce qu'on veut pour détecter des bords.

■ 2. Deux coordonnées, un indice : le ruban

Avant d'écrire le code, un outil : transformer deux coordonnées en un seul indice. Il sert pour **n'importe quelle grille stockée dans une liste** — le noyau 3×3 en est une, et une image en mémoire est exactement ça. La mémoire est une rangée de cases (cours 01) : une grille s'y range **ligne après ligne, bout à bout**, comme un ruban.

```

la grille a l'ecran (w = 4)          la meme grille en memoire
                                     (les lignes bout a bout)

+---+---+---+---+
| 0 | 1 | 2 | 3 |
+---+---+---+---+
| 4 | 5 | * | 7 |          ->      +---+---+---+---+---+---+---+---+---+
| 8 | 9 | 10| 11|          | 0 | 1 | 2 | 3 | 4 | 5 | * | 7 | 8 | 9 | 10| 11|
+---+---+---+---+          +---+---+---+---+---+---+---+---+---+
| 8 | 9 | 10| 11|
+---+---+---+---+

```

La case (x, y) est à l'indice $y * w + x$: sauter y lignes complètes ($y \times w$ cases), puis avancer de x . Le pixel $*$ en $(x = 2, y = 1)$: $1 \times 4 + 2 = 6$. Pour le noyau, la grille fait 3 de large et les coordonnées dx, dy partent de -1 , d'où le $+ 1$: $(dy + 1) * 3 + (dx + 1)$. Le même calcul resservira aux cours 11d, 11f et 11g — et c'est ce que `getColor(x, y)` fait sous le capot à chaque appel.

Les sommes se font en `float`, avec un `ofClamp` final : les poids négatifs donnent facilement des valeurs sous 0 ou au-dessus de 255 (cours 11).

■ 3. Étape 1 — `appliquerNoyau()` : quatre boucles imbriquées

Même mise en place qu'au cours 11 (`source` , `resultat` , un `int effet { 0 }` ;), plus la fonction centrale (annoncée dans le `.h` : `void appliquerNoyau(std::vector<float> noyau, float biais);`) :

```
void ofApp::appliquerNoyau(std::vector<float> noyau, float biais) {
    int w = source.getWidth();
    int h = source.getHeight();

    for (int y = 0; y < h; y++) {
        for (int x = 0; x < w; x++) {           // tous les pixels (cours 11)
            float r = biais, g = biais, b = biais;

            for (int dy = -1; dy <= 1; dy++) {   // les neuf voisins
                for (int dx = -1; dx <= 1; dx++) {
                    int px = ofClamp(x + dx, 0, w - 1);
                    int py = ofClamp(y + dy, 0, h - 1);
                    ofColor v = source.getColor(px, py);
                    float poids = noyau[(dy + 1) * 3 + (dx + 1)];
                    r += v.r * poids;
                    g += v.g * poids;
                    b += v.b * poids;
                }
            }
            resultat.setColor(x, y, ofColor(ofClamp(r, 0, 255),
                                             ofClamp(g, 0, 255), ofClamp(b, 0, 255))
        }
    }
    resultat.update();
}
```

Teste-la tout de suite avec le noyau identité — appelle dans `setup()` :

```
appliquerNoyau({ 0, 0, 0,
                 0, 1, 0,
                 0, 0, 0 }, 0);
```

L'image est copiée : le poids 1 au centre, 0 partout ailleurs. Ce qui se passe dans la fonction :

- Les deux boucles **extérieures** sont celles du cours 11 : tous les pixels. Les deux **intérieures** parcourent les voisins : `dx` et `dy` valent `-1`, `0` ou `1` — les neuf positions autour de `(x, y)`, `(0, 0)` étant le pixel lui-même. `r += ...` est le raccourci de `r = r + ...`.
- Le poids du voisin `(dx, dy)` est à l'indice `(dy + 1) * 3 + (dx + 1)` : le **ruban** de la section précédente, appliqué à la grille `3 × 3` du noyau (largeur 3, coordonnées décalées de `+1` car elles partent de `-1`).

- Les sommes se font en `float`, avec un `ofClamp` final : des poids négatifs sortent facilement de 0-255 (cours 11).
- La liste `{ 0, 0, 0, ... }` est créée **sur place**, dans l'appel — écrite sur trois lignes, elle ressemble à la grille qu'elle représente.

Le problème des bords, réglé au passage : le pixel `(0, 0)` n'a pas de voisin en `(-1, -1)`. Trois solutions classiques — ignorer les bords, les considérer noirs, ou **réutiliser le pixel du bord**. C'est la troisième qu'on prend, avec l'`ofClamp` sur `px / py` : simple, et sans liseré noir.

Essaie : déplace le 1 du noyau identité dans un coin — l'image se décale. Dans quel sens ? (La lecture à l'envers du cours 14 répond.)

■ 4. Étape 2 — les noyaux célèbres

Une fonction `calculer()` qui choisit le noyau selon `effet` :

```
switch (effet) {
case 1: appliquerNoyau({ 1/9.0f, 1/9.0f, 1/9.0f,      // flou boîte : la moyenne
                        1/9.0f, 1/9.0f, 1/9.0f,
                        1/9.0f, 1/9.0f, 1/9.0f }, 0); break;
case 2: appliquerNoyau({ 1/16.0f, 2/16.0f, 1/16.0f,   // flou gaussien : le centre pèse plus
                        2/16.0f, 4/16.0f, 2/16.0f,
                        1/16.0f, 2/16.0f, 1/16.0f }, 0); break;
case 3: appliquerNoyau({ 0, -1, 0,                    // netteté (somme = 1)
                        -1, 5, -1,
                        0, -1, 0 }, 0); break;
case 4: appliquerNoyau({ -1, -1, -1,                  // contours (somme = 0)
                        -1, 8, -1,
                        -1, -1, -1 }, 0); break;
case 5: appliquerNoyau({ -1, 0, 0,                    // relief : deux coins opposés
                        0, 0, 0,
                        0, 0, 1 }, 128); break;
}
```

Deux règles de lecture des noyaux :

- **Somme des poids = 1** : la luminosité est conservée (flou, netteté). **Somme = 0** : les zones plates deviennent noires — seules les **différences** restent (contours).
- Le **biais** (dernier paramètre) : le relief calcule une différence, aussi souvent négative que positive — sans rien, la moitié de l'image serait noire. Le biais est ajouté avant de borner : avec 128, une différence nulle donne un gris moyen, les positives éclaircissent, les négatives assombrissent. Un relief éclairé en biais.

Essaie : chaque noyau, un par un, sur l'image. Puis fabrique le tien — que fait `{ 0, 0, 0, 1, 0, -1, 0, 0, 0 }` avec un biais de 128 ?

■ 5. Étape 3 — le drapeau : ne payer qu'une fois

400 000 pixels × 9 voisins = 3,6 millions de lectures — trop pour recommencer 60 fois par seconde. Or le résultat ne change que quand on change d'effet. Dans le `.h` : `bool aRecalculer { true }` ;, et :

```
void ofApp::update() {
    if (aRecalculer) {
        calculer();
        aRecalculer = false;
    }
}

void ofApp::keyPressed(int key) {
    if (key >= '0' && key <= '9') {
        effet = key - '0';
        aRecalculer = true;
    }
}
```

Un `bool` qui dit « il y a du travail à faire », levé par l'événement, rabaissé une fois le travail fait : c'est un **drapeau**. Motif très courant pour tout ce qui coûte cher — tu le reverras au cours 18.

Essaie : enlève le drapeau (recalcule à chaque frame) et compare la fluidité. Puis remets-le.

■ Exercices

1. **Lire avant de lancer** — que fait le noyau `{ 0, 0, 0, 0, 2, 0, 0, 0, 0 }` (biais 0) ? Et `{ 0, 0, 0, 0, 0, 0, 0, 1 }` ? Réponds, puis vérifie.
2. **Le dessin au crayon** — le **négatif** des contours (reprennds le négatif du cours 11, appliqué au résultat du noyau contours) : traits sombres sur fond clair.
3. **Le double flou** — applique le flou deux fois de suite. Il faut copier `resultat` dans `source` entre les deux passes (ou une troisième image). Compare avec le flou gaussien simple.
4. **Sobel** (*plus costaud*) — le noyau `{ -1, -2, -1, 0, 0, 0, 1, 2, 1 }` détecte les contours **horizontaux** ; le même tourné d'un quart de tour, les verticaux. Applique les deux, puis combine : $\sqrt{h*h + v*v}$ sur chaque canal — LE détecteur de contours, celui qui ressort au cours 18 pour suivre les formes.

■ Ce qu'il faut retenir

- Une convolution : chaque pixel devient la **moyenne pondérée** de ses neuf voisins ; la grille de poids s'appelle un **noyau**.
- Flou, netteté, contours, relief : le même code, neuf nombres qui changent.
- Somme des poids = 1 → luminosité conservée ; = 0 → seules les différences restent ; le **biais** recentre un résultat signé.
- L'image en mémoire est un **ruban** : la case `(x, y)` est à l'indice `y * w + x` — valable pour toute grille rangée dans une liste.
- Les bords : `ofClamp` sur les coordonnées des voisins — le bord se réutilise.
- Le **drapeau** : un `bool` levé par l'événement, rabaissé après le travail — ne payer les calculs lourds qu'une fois.